

Módulo 9: APIs web con FastAPI (Automatización)

Contenidos clave

Estructura de proyecto, routers y dependencias:

- La estructura de proyecto se refiere a que no conviene poner todos los endpoints en un solo archivo. En lugar de:

```
main.py
├── GET users
├── POST users
├── GET orders
├── POST orders
├── DELETE orders
└── login
```

se organiza así ->

```
app/
├── main.py
├── routers/
│   ├── users.py
│   └── orders.py
├── schemas/
│   └── order.py
├── services/
├── models/
└── database.py
```

Lo anterior facilita mantener y probar el proyecto.

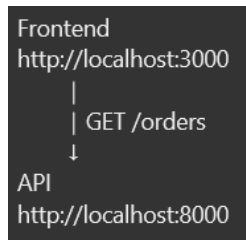
- Router: un router agrupa endpoints relacionados.
- Dependencias: FastAPI permite definir cosas que varios endpoints necesitan y reutilizarlas. Un ejemplo es el obtener una sesión de base de datos: `Depends(get_db)` le dice a FastAPI algo cómo “antes de ejecutar este endpoint, necesito obtener una sesión de base de datos”, esto evita repetir la creación/cierre de sesiones en cada endpoint.

Esquemas Pydantic, validación y OpenAPI:

- En FastAPI, Pydantic se utiliza para definir qué datos puede recibir y devolver la API.
- FastAPI genera automáticamente documentación de la API mediante OpenAPI. Al ejecutar la aplicación normalmente puedes acceder a `/docs` y obtener Swagger UI, y se pueden probar los endpoints directamente desde la documentación.

Autenticación JWT, middlewares, CORS:

- JWT (JSON Web Token) Se utiliza para identificar/autenticar al usuario después de iniciar sesión. Flujo: Usuario (username + password) -> POST / login -> FastAPI valida credenciales -> JWT -> Cliente guarda token, después para acceder a un endpoint protegido:
GET / orders
Authorization: Bearer <token>
FastAPI valida el token antes de permitir el acceso.
- Middleware: es un código que puede ejecutarse alrededor de las peticiones HTTP.
Request -> Middleware -> Endpoint -> Response <- Middleware <- Cliente.
Se puede utilizar para cosas como: logging, medir tiempo de respuesta, agregar headers, controlar ciertas solicitudes y procesamiento común.
- CORS (Cross-Origin Resource Sharing). Es una política relacionada con permitir que un frontend ubicado en un origen pueda hacer peticiones a tu API desde otro origen.



Son orígenes diferentes, pero se puede configurar FastAPI para permitir explícitamente solicitudes provenientes de ese frontend. En producción es mejor especificar los orígenes permitidos.

Testing de endpoints con pytest + aiohttp: Probar la API completa.

- pytest: framework de pruebas.
- aiohttp: permite realizar peticiones HTTP de forma asíncrona.

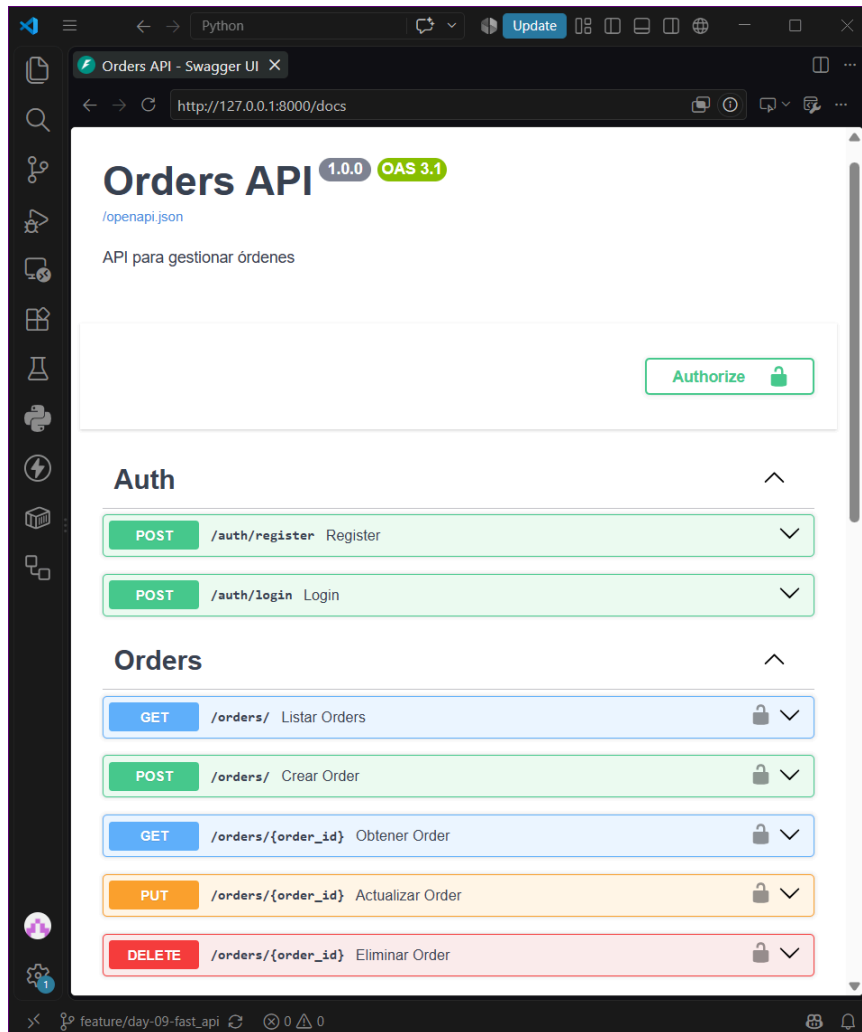
Objetivos

- Exponer una API coherente y validada.

Se utilizó:

- FastAPI, para organizar la aplicación en routers y exponer endpoints REST.
- Routers separados para autenticación y órdenes.
- SQLAlchemy, para persistir usuarios, órdenes y elementos.
- Pydantic, utilizado en [schemas.py](#) para validar:
 - Estados de las órdenes
 - Nombre de productos
 - Cantidades mayores que cero
 - Precios positivos
 - Listas de elementos obligatorias
 - Emails y contraseñas de usuarios
- OpenAPI/Swagger, generado automáticamente por FastAPI para consultar y probar los endpoints.

La API quedó organizada con operaciones claras:



- Asegurar endpoints y documentarlos.

- Login basado en JWT en: POST / auth/login
- Hash seguro de contraseñas usando pwplib
- Dependencia get_current_user para validar el token
- Protección de los endpoints de órdenes mediante: Depends(get_current_user)
- Aislamiento de datos por usuario mediante filtros como: Order.user_id == current_user.id
- Respuesta 401 Unauthorized cuando no se proporciona un token válido
- Documentación automática mediante Swagger/OpenAPI
- Configuración de CORS para permitir el consumo desde aplicaciones frontend
- Middleware HTTP para agregar el tiempo de procesamiento

Laboratorio

- CRUD de Orders con validación.

Implementación de los endpoints:

- POST /orders/: Crear una orden.

POST

/orders/ Crear Order

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value

Schema

```
{
  "status": "PENDING",
  "items": [
    {
      "product_name": "Teclado mecánico",
      "quantity": 2,
      "unit_price": 35.5
    },
    {
      "product_name": "Mouse",
      "quantity": 1,
      "unit_price": 20
    }
  ]
}
```

Execute

Clear

201

Response body

```
{
  "id": 2,
  "user_id": 1,
  "status": "PENDING",
  "created_at": "2026-09-23T20:45:29.912419",
  "items": [
    {
      "id": 2,
      "product_name": "Teclado mecánico",
      "quantity": 2,
      "unit_price": "35.50"
    },
    {
      "id": 3,
      "product_name": "Mouse",
      "quantity": 1,
      "unit_price": "20.00"
    }
  ]
}
```

Download

Response headers

```
content-length: 235
content-type: application/json
date: Wed, 23 Sep 2026 20:45:28 GMT
server: uvicorn
```

Responses

Code	Description	Links
201	Successful Response	No links

Media type

application/json

Controls Accept header.

```
INFO: 127.0.0.1:50651 - "POST /orders/ HTTP/1.1" 201 Created
INFO: 127.0.0.1:47857 - "POST /orders/ HTTP/1.1" 201 Created
```

- GET /orders/: Listar órdenes.

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/orders/' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/orders/
```

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "product_name": "string",
  "quantity": 1,
  "unit_price": "1.00"
},
{
  "id": 2,
  "user_id": 1,
  "status": "PENDING",
  "created_at": "2026-09-23T20:45:29.912419",
  "items": [
    {
      "id": 2,
      "product_name": "Teclado mecánico",
      "quantity": 2,
      "unit_price": "35.50"
    },
    {
      "id": 3,
      "product_name": "Mouse",
      "quantity": 1,
      "unit_price": "20.00"
    }
  ]
}
]
```

Response headers

```
content-length: 395
content-type: application/json
```

INFO: 127.0.0.1:58074 - "GET /orders/ HTTP/1.1" 200 OK

- GET /orders/{id}: Obtener una orden.
- PUT /orders/{id}: Actualizar una orden.

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/orders/2' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "status": "CONFIRMED"
  }'
```

Request URL

http://127.0.0.1:8000/orders/2

Server response

Code

Details

200

Response body

```
{
  "id": 2,
  "user_id": 1,
  "status": "CONFIRMED",
  "created_at": "2026-09-23T20:45:29.912419",
  "items": [
    {
      "id": 2,
      "product_name": "Teclado mecánico",
      "quantity": 2,
      "unit_price": "35.50"
    },
    {
      "id": 3,
      "product_name": "Mouse",
      "quantity": 1,
      "unit_price": "20.00"
    }
  ]
}
```

Download

Response headers

```
content-length: 237
content-type: application/json
date: Wed, 23 Sep 2026 20:52:31 GMT
server: uvicorn
```

Responses

INFO:

127.0.0.1:38037 - "PUT /orders/2 HTTP/1.1" 200 OK

- DELETE /orders/{id}: Eliminar una orden.

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/orders/2' \
  -H 'accept: */*'
```

Request URL

http://127.0.0.1:8000/orders/2

Server response

Code

Details

204

Response headers

```
content-type: application/json
date: Wed, 23 Sep 2026 20:56:15 GMT
server: uvicorn
```

Responses

Code	Description	Links
204	Successful Response	No links
422	Validation Error	No links

Media type

application/json

Example Value

Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
    }
  ],
}
```

INFO: 127.0.0.1:2967 - "DELETE /orders/2 HTTP/1.1" 204 No Content

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/orders/2' \
-H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/orders/2
```

Server response

Code	Details
404 <i>Undocumented</i>	Error: Not Found Response body <pre>{ "detail": "Order no encontrada" }</pre> Response headers <pre>content-length: 32 content-type: application/json date: Wed, 23 Sep 2026 20:58:22 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type: Controls Accept header.

[Example Value](#) | [Schema](#)

- Login JWT básico.

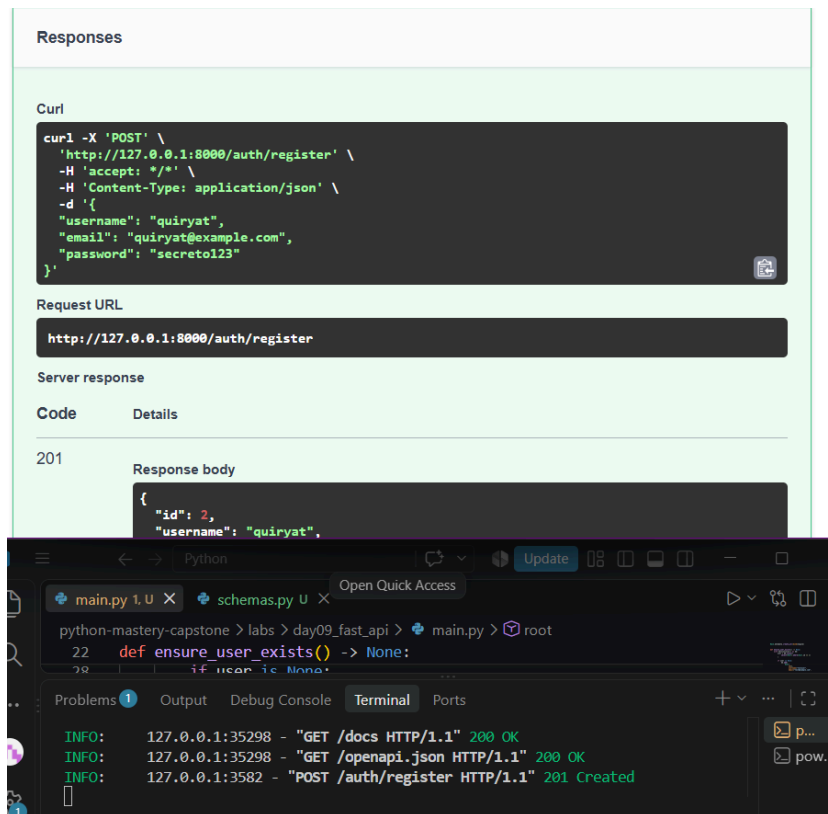
Se implementaron:

- POST /auth/register
- POST /auth/login

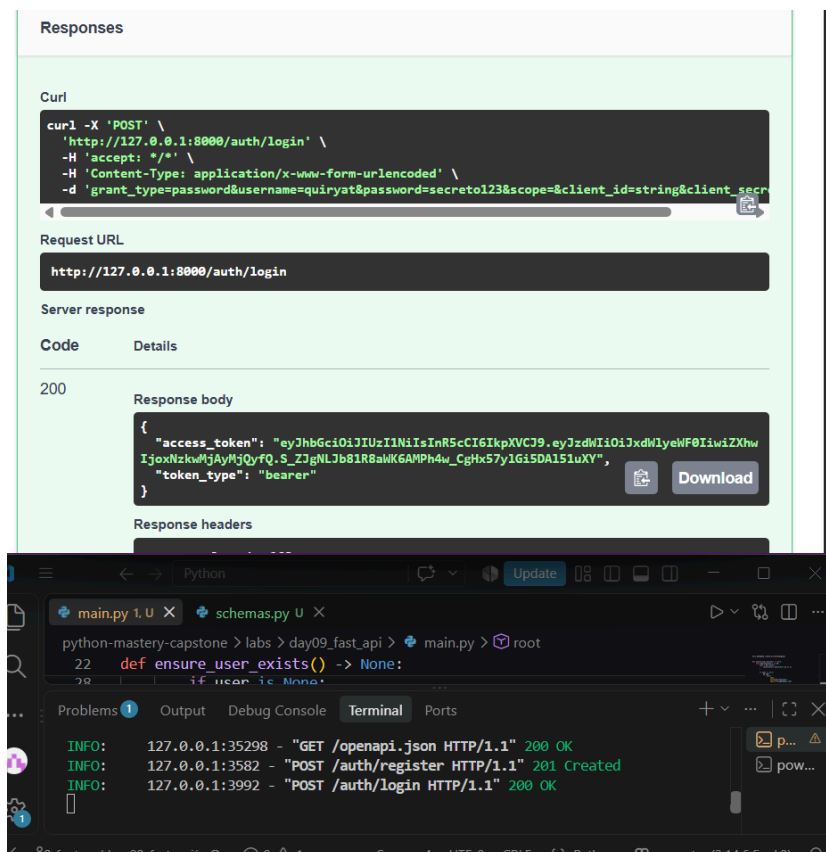
Authorize 		
Auth ^		
POST	/auth/register Register	▼
POST	/auth/login Login	▼
Orders ^		
GET	/orders/ Listar Orders	 ▼
POST	/orders/ Crear Order	 ▼
GET	/orders/{order_id} Obtener Order	 ▼
PUT	/orders/{order_id} Actualizar Order	 ▼
DELETE	/orders/{order_id} Eliminar Order	 ▼
default ^		
GET	/ Root	▼

El flujo fue:

1. Registrar un usuario



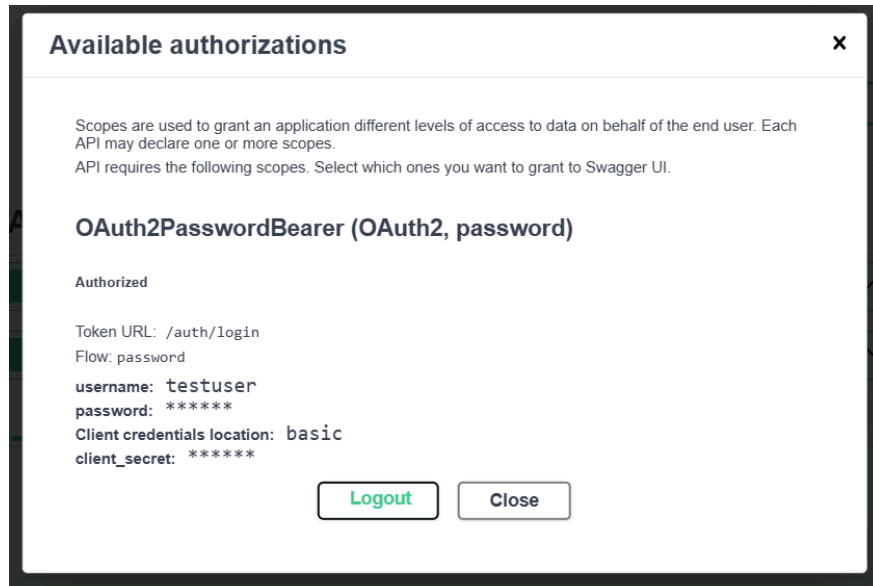
2. Cifrar su contraseña
3. Iniciar sesión



4. Generar un token JWT

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJxdWlyeWF0IiwiaXhwIjoxNzkwMjAyMjQyYQ.S_ZjNLJb81R8aWK6AMPh4w_CgHx57yl6i5DA151uXY",
  "token_type": "bearer"
}
```

5. Enviar el token como Bearer



6. Validar el token antes de acceder a /orders/

También se comprobó que un usuario no pudiera consultar las órdenes de otro.

Responses		
Curl <pre>curl -X 'GET' \ 'http://127.0.0.1:8000/orders/' \ -H 'accept: */*'</pre>		
Request URL <pre>http://127.0.0.1:8000/orders/</pre>		
Server response		
Code	Details	
401	Error: Unauthorized	
<i>Undocumented</i>	Response body <pre>{ "detail": "Not authenticated" }</pre>  Download	
	Response headers <pre>content-length: 30 content-type: application/json date: Wed, 23 Sep 2026 23:05:48 GMT server: uvicorn www-authenticate: Bearer</pre>	
Responses		
Code	Description	Links
200	Successful Response	No links

- Tests de integración con DB temporal.

Implementación de una base SQLite en memoria para los tests mediante: aqlite:// y StaticPool, además de sobrescribir la dependencia de base de datos con: **app.dependency_overrides[get_db]**.

Verificación de:

- Endpoint raíz
- Registro de usuarios
- Usuarios duplicados
- Login correcto
- Login incorrecto
- Protección sin token
- Creación de órdenes
- Listado de órdenes
- Aislamiento entre usuarios
- Actualización
- Eliminación.

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run pytest -q
..... [100%]
===== warnings summary =====
..\..\AppData\Local\pypoetry\Cache\virtualenvs\python-mastery-capstone-Sh3whZ1x-py3.14\Lib\site-packages\_pytest\config\_
C:\Users\Quiryat\AppData\Local\pypoetry\Cache\virtualenvs\python-mastery-capstone-Sh3whZ1x-py3.14\Lib\site-packages\_pytest
o_mode

    self._warn_or_fail_if_strict(f"Unknown config option: {key}\n")

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
15 passed, 1 warning in 1.77s
```